## File and Directory Operations:

1. `ls` - List files in a directory.
   - `ls -l` (long listing with details)
   - `ls -a` (show hidden files)
2. `cd [directory]` - Change directory.
3. `pwd` - Print working directory.
4. `mkdir [directory]` - Create a new directory.
5. `rmdir [directory]` - Remove an empty directory.
6. `cp [source] [destination]` - Copy files or directories.
7. `mv [source] [destination]` - Move/rename files or directories.
8. `rm [file]` - Remove files.
   - `rm -r [directory]` (recursively remove a directory)
9. `touch [file]` - Create an empty file or update its timestamp.
10. `cat [file]` - Concatenate and display file contents.
11. `more [file] / less [file]` - View file contents page by page.
12. `head [file]` - Display the first 10 lines of a file.
13. `tail [file]` - Display the last 10 lines of a file.
    - `tail -f [file]` (follow file updates)

## File Permissions:

1. `chmod [permissions] [file]` - Change file permissions.

- `chmod 755 [file]` (set read/write/execute for owner, read/execute for others)

2. `chown [owner]:[group] [file]` - Change file owner and group.

## System Information:

1. `whoami` - Display the current user.
2. `uname -a` - Show system information.
3. `df -h` - Display disk space usage.
4. `du -h [directory]` - Estimate file space usage.
5. `free -m` - Show memory and swap usage.
6. `top` - Display running processes.
7. `ps` - Display snapshot of current processes.
8. `uptime` - Show how long the system has been running.
9. `date` - Display current date and time.

## Process Management:

1. `ps aux` - Show all running processes.
2. `kill [PID]` - Terminate a process by its PID.
3. `killall [process_name]` - Terminate all processes with the given name.
4. `bg` - Resume a suspended job in the background.
5. `fg` - Bring a background job to the foreground.

6. `jobs` - List background jobs.
7. `top` - Display real-time system resource usage.
8. `nice [command]` - Start a process with a different priority.
9. `nohup [command]` - Run a command immune to hangups.

## Networking:

1. `ping [hostname/IP]` - Send ICMP echo requests to test network connectivity.
2. `ifconfig / ip a` - Display network interfaces and IP addresses.
3. `netstat -an` - Show network connections and listening ports.
4. `curl [URL]` - Retrieve data from or send data to a URL.
5. `wget [URL]` - Download files from the internet.

## Compression and Archiving:

1. `tar -cvf [archive.tar] [file/dir]` - Create a tar archive.
2. `tar -xvf [archive.tar]` - Extract a tar archive.
3. `gzip [file]` - Compress a file.
4. `gunzip [file.gz]` - Decompress a `.gz` file.
5. `zip [archive.zip] [file/dir]` - Create a zip archive.
6. `unzip [archive.zip]` - Extract a zip archive.

## Searching and Finding Files:

1. `find [directory] -name [pattern]` - Search for files matching a pattern.
2. `grep [pattern] [file]` - Search for a pattern in a file.

- `grep -r [pattern] [directory]` (recursively search in a directory)

3. `locate [filename]` - Find the location of a file.
4. `which [command]` - Show the path of a command.

## Text Manipulation:

1. `echo [text]` - Output text to the terminal.
2. `sort [file]` - Sort lines in a file.
3. `uniq [file]` - Remove duplicate lines from a sorted file.
4. `wc [file]` - Count lines, words, and characters in a file.
5. `cut -d [delimiter] -f [fields] [file]` - Extract fields from a file.
6. `sed 's/[pattern]/[replacement]/g' [file]` - Find and replace text in a file.
7. `awk '{print $1, $2}' [file]` - Pattern scanning and processing language.

## Scripting and Execution:

1. `bash [script.sh]` - Execute a Bash script.
2. `source [script.sh]` - Execute a script in the current shell.
3. `./[script.sh]` - Run a script with execution permission.
4. `alias [alias_name]='[command]'` - Create a command alias.

## Environment Variables:

1. `export [VARIABLE]=[value]` - Set environment variables.
2. `echo $[VARIABLE]` - Print the value of a variable.
3. `env` - List environment variables.
4. `unset [VARIABLE]` - Unset environment variables.

### Miscellaneous:

1. `history` - Show command history.
2. `clear` - Clear the terminal screen.
3. `man [command]` - Display manual for a command.
4. `sleep [seconds]` - Pause for a specified time.
5. `time [command]` - Measure the time taken by a command.

Here is a more specific and comprehensive list of Bash commands related to file handling and management:

## 1. File and Directory Listing:

- `ls` – List directory contents.
- `ls -l` – Long listing format (shows file permissions, owner, size, etc.).
- `ls -a` – List all files, including hidden files (those starting with `.`).
- `ls -lh` – List with human-readable sizes.
- `ls -R` – Recursively list subdirectories.
- `ls -lt` – List files sorted by modification time.

## 2. File Creation:

- `touch [file]` – Create an empty file or update its timestamp.
- `> [file]` – Create an empty file or overwrite an existing file with no content.

## 3. File Copying:

- `cp [source] [destination]` – Copy files.
- `cp file1 file2` – Copy `file1` to `file2`.
- `cp -r [directory] [destination]` – Recursively copy directories.
- `cp -u [source] [destination]` – Copy files only if the source is newer than the destination.

## 4. File Moving and Renaming:

- `mv [source] [destination]` – Move or rename files and directories.
- `mv file1 newfile` – Rename `file1` to `newfile`.
- `mv file1 /path/to/directory/` – Move `file1` to the specified directory.
- `mv -u [source] [destination]` – Move files only if the source is newer than the destination.

## 5. File Deletion:

- `rm [file]` – Remove a file.
- `rm -i [file]` – Interactive removal (ask for confirmation).
- `rm -f [file]` – Force remove a file without asking for confirmation.

- `rm -r [directory]` – Recursively delete a directory and its contents.
- `rm -rf [directory]` – Force remove a directory and its contents.

## 6. Directory Management:

- `mkdir [directory]` – Create a new directory.
- `mkdir -p [directory]` – Create a directory and its parent directories if they don't exist.
- `rmdir [directory]` – Remove an empty directory.
- `rm -r [directory]` – Remove a directory and all its contents.

## 7. File Permissions:

- `chmod [permissions] [file]` – Change file permissions.
- `chmod 755 [file]` – Set read, write, and execute for the owner, and read and execute for others.
- `chmod +x [file]` – Add execute permissions to a file.
- `chown [owner] [file]` – Change file owner.
- `chown user:group [file]` – Change file owner and group.
- `chgrp [group] [file]` – Change group ownership of a file.

## 8. File Viewing:

- `cat [file]` – View the contents of a file.
- `cat file1 file2` – Concatenate `file1` and `file2`, then display the output.

- `more [file]` – View the file contents, one page at a time.
- `less [file]` – Similar to `more`, but allows backward movement in the file.
- `head [file]` – Display the first 10 lines of a file.
- `head -n [num] [file]` – Display the first `[num]` lines of a file.
- `tail [file]` – Display the last 10 lines of a file.
- `tail -f [file]` – Continuously monitor the file for new data.

## 9. File Searching:

- `find [path] -name [filename]` – Search for files by name.
- `find /home -name "*.txt"` – Find all `.txt` files in the `/home` directory.
- `grep [pattern] [file]` – Search for a pattern inside files.
- `grep -r [pattern] [directory]` – Recursively search for a pattern in a directory.
- `locate [filename]` – Find files by name quickly using a pre-built database.
- `which [command]` – Show the full path of a command.

## 10. File Comparison:

- `diff [file1] [file2]` – Compare two files and display their differences.
- `diff -u file1 file2` – Show differences with unified format.
- `cmp [file1] [file2]` – Compare two files byte by byte.
- `comm [file1] [file2]` – Compare two sorted files line by line.

## 11. File Compression and Archiving:

- `tar -cvf [archive.tar] [file/dir]` – Create a tar archive.
- `tar -xvf [archive.tar]` – Extract a tar archive.
- `gzip [file]` – Compress a file using gzip.
- `gunzip [file.gz]` – Decompress a `.gz` file.
- `zip [archive.zip] [file/dir]` – Create a zip archive.
- `unzip [archive.zip]` – Extract a zip archive.

## 12. File Size and Disk Space:

- `du [file/dir]` – Estimate file space usage.
- `du -h [dir]` – Show file sizes in human-readable format.
- `du -sh [dir]` – Show only the total size of a directory.
- `df -h` – Display disk space usage in human-readable format.

## 13. File Ownership and Attributes:

- `ls -l` – View the file's owner and permissions.
- `chattr [attributes] [file]` – Change file attributes.
- `chattr +i [file]` – Make the file immutable (cannot be modified, deleted, renamed).
- `lsattr [file]` – List file attributes.

## 14. File Links:

- `ln [target] [link]` – Create a hard link to a file.

- `ln -s [target] [link]` – Create a symbolic (soft) link to a file.

## 15. Backup and Restore:

- `rsync [options] [source] [destination]` – Sync files and directories.
- `rsync -av [source] [destination]` – Archive mode, verbose.
- `rsync -avz [source] [destination]` – Compress data during transfer.

Here's a comprehensive list of Bash commands and utilities specifically for string processing and manipulation:

## 1. Basic String Operations:

- **Variable Assignment**:
- `str="Hello World"` – Assign a string to a variable.
- **Printing Strings**:
- `echo $str` – Print the value of a string variable.
- `printf "%s\n" "$str"` – Print formatted string (more control over formatting than `echo`).

## 2. String Length:

- `echo ${#str}` – Get the length of a string.
- Example: `str="Hello World"; echo ${#str}` outputs `11`.

## 3. String Substring:

- `echo ${str:position:length}` – Extract a substring.

- Example: `str="Hello World"; echo ${str:6:5}` outputs `World`.
- `echo ${str:position}` – Extract a substring from the specified position to the end of the string.
- Example: `echo ${str:6}` outputs `World`.

## 4. String Substitution/Replacement:

- **Replace first match**:
- `echo ${str/substring/replacement}` – Replace the first occurrence of `substring` with `replacement`.
- Example: `str="Hello World"; echo ${str/World/Everyone}` outputs `Hello Everyone`.
- **Replace all matches**:
- `echo ${str//substring/replacement}` – Replace all occurrences of `substring` with `replacement`.
- Example: `str="Hello World World"; echo ${str//World/Everyone}` outputs `Hello Everyone Everyone`.
- **Delete substring**:
- `echo ${str/substring/}` – Delete the first occurrence of `substring`.
- `echo ${str//substring/}` – Delete all occurrences of `substring`.

## 5. String Case Conversion:

- **Convert to uppercase**:
- `echo ${str^^}` – Convert entire string to uppercase.

- Example: `str="hello"; echo ${str^^}` outputs `HELLO`.
- **Convert to lowercase**:
- `echo ${str,,}` – Convert entire string to lowercase.
- Example: `str="HELLO"; echo ${str,,}` outputs `hello`.
- **Uppercase first letter**:
- `echo ${str^}` – Capitalize the first letter.
- Example: `str="hello"; echo ${str^}` outputs `Hello`.
- **Lowercase first letter**:
- `echo ${str,}` – Convert the first letter to lowercase.
- Example: `str="HELLO"; echo ${str,}` outputs `hELLO`.

## 6. String Searching:

- **Find substring**:
- `[[ $str == *substring* ]]` – Check if a string contains a substring.
- Example: `[[ $str == *World* ]] && echo "Found"` outputs `Found` if `World` is present in `str`.
- **Extract substring using pattern matching**:
- `echo ${str#pattern}` – Remove the shortest match of `pattern` from the start of the string.
- Example: `str="Hello World"; echo ${str#Hello }` outputs `World`.
- `echo ${str##pattern}` – Remove the longest match of `pattern` from the start of the string.

- `echo ${str%pattern}` – Remove the shortest match of `pattern` from the end of the string.
- Example: `str="Hello World"; echo ${str% World}` outputs `Hello`.
- `echo ${str%%pattern}` – Remove the longest match of `pattern` from the end of the string.

## 7. String Concatenation:

- Concatenate two strings:
- `str1="Hello"; str2="World"; result="$str1 $str2"; echo $result` outputs `Hello World`.

## 8. String Splitting:

- **Split a string by a delimiter**:
- Use `IFS` (Internal Field Separator):

```
str="apple,orange,banana"

IFS=',' read -ra arr <<< "$str"

echo "${arr[0]}"  # Outputs: apple

echo "${arr[1]}"  # Outputs: orange
```

- `IFS` is set to `,` to split the string, and `read -ra` creates an array from the split parts.

## 9. Pattern Matching with `grep`:

- **Search for a pattern**:
- `grep [pattern] [file]` – Search for lines in a file matching a pattern.
- Example: `grep "Hello" file.txt` – Search for `Hello` in `file.txt`.
- `grep -i [pattern] [file]` – Case-insensitive search.
- `grep -r [pattern] [directory]` – Recursively search for a pattern in a directory.

## 10. Text Processing with `sed`:

- **Find and Replace**:
- `sed 's/pattern/replacement/' file` – Replace the first occurrence of a pattern in each line.
- Example: `sed 's/Hello/Hi/' file.txt` replaces the first `Hello` with `Hi`.
- `sed 's/pattern/replacement/g' file` – Replace all occurrences of a pattern in each line.
- **Delete lines**:
- `sed '/pattern/d' file` – Delete lines matching a pattern.
- **Print specific lines**:
- `sed -n '5p' file` – Print the 5th line of a file.

## 11. Text Processing with `awk`:

- **Print specific fields**:
- `awk '{print $1, $3}' file` – Print the 1st and 3rd columns (fields) from each line in a file.

- Example: `echo "a b c" | awk '{print $1}'` outputs `a`.
- **Conditional printing**:
- `awk '$1 ~ /pattern/ {print $0}' file` – Print lines where the 1st field matches a pattern.
- `awk '$3 > 50 {print $0}' file` – Print lines where the 3rd field is greater than 50.
- **Text calculations**:
- `awk '{sum += $3} END {print sum}' file` – Sum up the 3rd field for all lines.

## 12. String Sorting and Uniqueness:

- **Sort**:
- `sort file` – Sort lines in a file alphabetically.
- `sort -n file` – Sort lines numerically.
- `sort -r file` – Reverse sort.
- **Remove Duplicates**:
- `uniq file` – Remove adjacent duplicate lines.
- `sort file | uniq` – Remove duplicate lines after sorting.

## 13. Word Count ( `wc` ):

- **Count number of lines, words, and characters**:
- `wc file` – Display the line count, word count, and character count of a file.
- `wc -l` – Count the number of lines.

- `wc -w` – Count the number of words.
- `wc -c` – Count the number of characters.

## 14. String Reversal:

- **Reverse a string**:
- `echo $str | rev` – Reverse the string.

## 15. Regular Expressions:

- **Basic Regular Expressions with `grep`** :
- `grep "pattern" file` – Search for a basic regular expression.
- `.` – Matches any character.
- `^` – Anchors the match at the beginning of a line.
- `$` – Anchors the match at the end of a line.
- `[abc]` – Matches any character in the set `a`, `b`, or `c`.
- **Extended Regular Expressions with `grep -E` or `egrep`** :
- `|` – Logical OR between patterns.
- `+` – Matches one or more occurrences of the previous element.
- `?` – Matches zero or one occurrence of the previous element.

These string processing techniques are powerful when working with text in Bash scripts or the command line, especially for file parsing, text manipulation, or simple automation tasks. Let me know if you want more examples for specific use cases!

Here is a list of basic Bash scripting commands, including loops, conditional statements,

and other foundational coding constructs. These are the building blocks of writing Bash scripts.

## 1. Variables:

- **Defining a variable**:

```
variable_name="value"
```

Example:

```
name="John"
echo $name
```

## 2. Conditional Statements ( `if` ):

- **Basic `if` structure**:

```
if [ condition ]; then
# commands if true
else
# commands if false
fi
```

- **Example: Check if a file exists**:

```
if [ -f "/path/to/file" ]; then
echo "File exists."
else
echo "File does not exist."
fi
```

- **Example: Compare numbers**:

```
num=10
if [ $num -gt 5 ]; then
echo "Number is greater than 5."
else
echo "Number is not greater than 5."
fi
```

- **Operators for comparisons**:
- Numeric comparison:
- `-eq` – equal to
- `-ne` – not equal to
- `-lt` – less than
- `-le` – less than or equal to

- `-gt` – greater than
- `-ge` – greater than or equal to
- String comparison:
- `==` – equal to
- `!=` – not equal to
- `-z` – string is null (zero length)
- **Example: String comparison**:

```
str="Hello"
if [ "$str" == "Hello" ]; then
echo "The string is Hello."
else
echo "The string is not Hello."
fi
```

### 3. `elif` (Else If):

- **Example of `elif` in a conditional structure**:

```
num=10
if [ $num -lt 5 ]; then
echo "Number is less than 5."
```

```
elif [ $num -eq 10 ]; then

echo "Number is 10."

else

echo "Number is greater than 5 and not 10."

fi
```

## 4. Loops:

### `for` Loop:

- **Basic `for` loop**:

```
for item in list; do

# commands

done
```

- **Example: Iterate over a list of items**:

```
for i in 1 2 3 4 5; do

echo "Number: $i"

done
```

- **Example: Loop through files in a directory**:

```
for file in /path/to/directory/*; do
echo "File: $file"
done
```

### `while` Loop:

- **Basic `while` loop**:

```
while [ condition ]; do
# commands
done
```

- **Example: Print numbers from 1 to 5**:

```
count=1
while [ $count -le 5 ]; do
echo "Count: $count"
((count++))
done
```

### `until` Loop:

- **`until` loop runs until a condition is true**:

```
until [ condition ]; do
# commands
done
```

- **Example: Use `until` loop**:

```
count=1
until [ $count -gt 5 ]; do
echo "Count: $count"
((count++))
done
```

## 5. Case Statement:

- **`case` structure is similar to `switch` in other languages**:

```
case $variable in
pattern1)
# commands
;;
pattern2)
# commands
;;
```

```
*)
# default case
;;
esac
```

- **Example: Simple case statement**:

```
option="A"
case $option in
"A")
echo "Option A selected."
;;
"B")
echo "Option B selected."
;;
*)
echo "Invalid option."
;;
esac
```

## 6. Functions:

- **Defining a function**:

```
function_name() {
# commands
}
```

- **Example: Simple function**:

```
greet() {
echo "Hello, $1!"
}

greet "John"
```

- **Calling a function**:

```
function_name arguments
```

## 7. Arithmetic Operations:

- **Using `expr`**:

```
result=$(expr 5 + 3)
echo $result
```

- **Using `$(( ))` for arithmetic**:

```
result=$((5 + 3))
echo $result
```

- **Basic arithmetic operators**:
- `+` – Addition
- `-` – Subtraction
- `*` – Multiplication
- `/` – Division
- `%` – Modulo (remainder)
- **Example: Simple arithmetic**:

```
num1=10
num2=5
sum=$((num1 + num2))
echo "Sum: $sum"
```

## 8. Read User Input:

- **Use `read` to get input from the user**:

```
echo "Enter your name:"

read name

echo "Hello, $name!"
```

## 9. File Testing Operators:

- **File conditions**:
- `-e [file]` – Check if file exists.
- `-f [file]` – Check if it is a regular file.
- `-d [directory]` – Check if directory exists.
- `-r [file]` – Check if file is readable.
- `-w [file]` – Check if file is writable.
- `-x [file]` – Check if file is executable.
- `-s [file]` – Check if file has non-zero size.
- **Example: Check if file exists**:

```
if [ -e "file.txt" ]; then

echo "File exists."

else

echo "File does not exist."

fi
```

## 10. Logical Operators:

- **AND ( `&&` ) and OR ( `||` ) conditions in `if` statements**:
- **AND**:

```
if [ condition1 ] && [ condition2 ]; then
# commands
fi
```

- **OR**:

```
if [ condition1 ] || [ condition2 ]; then
# commands
fi
```

- **Example: Using AND operator**:

```
num=10
if [ $num -gt 5 ] && [ $num -lt 20 ]; then
echo "Number is between 5 and 20."
fi
```

## 11. Exit Status:

- **Check the exit status of the last command**:
- `$?` – Gives the exit status of the last executed command.
- `0` means the command was successful, any other value indicates an error.
- **Example**:

```
ls /nonexistent
if [ $? -ne 0 ]; then
echo "Error: Directory does not exist."
fi
```

## 12. Comments:

- **Single-line comments**:

```
# This is a comment
```

## 13. Script Arguments:

- **Access arguments passed to the script**:
- `$0` – Name of the script.
- `$1`, `$2`, `...` – First, second, etc., argument.

- `$#` – Number of arguments.
- `$@` – All arguments.
- **Example: Use arguments**:

```
echo "Script name: $0"
echo "First argument: $1"
echo "All arguments: $@"
```

This collection of basic Bash scripting commands provides a solid foundation for writing your own scripts and automating tasks. Let me know if you'd like further explanations or examples!

Here is an additional section with useful miscellaneous Bash commands related to date and time, file search, size checking, and more:

## 1. Date and Time:

- **Display current date and time**:

```
date
```

Example output: `Tue Sep 17 13:12:15 UTC 2024`

- **Format the date**:

```
date +"%Y-%m-%d %H:%M:%S"
```

Example output: `2024-09-17 13:12:15`

- **Store current date in a variable**:

```
current_date=$(date +"%Y-%m-%d")
echo $current_date
```

- **Get the number of seconds since the Unix epoch (Jan 1, 1970)**:

```
date +%s
```

- **Add or subtract days from the current date**:

```
date -d "+2 days"  # 2 days from today
date -d "-3 days"  # 3 days ago
```

## 2. File Search:

- **Find files by name**:

```
find /path/to/directory -name "filename"
```

Example: Find all `.txt` files:

```
find /home/user -name "*.txt"
```

- **Find files modified in the last N days**:

```
find /path/to/directory -mtime -N
```

Example: Find files modified in the last 7 days:

```
find /home/user -mtime -7
```

- **Find files larger than a certain size**:

```
find /path/to/directory -size +N[c]
```

Example: Find files larger than 10MB:

```
find /home/user -size +10M
```

- **Search for a pattern inside files ( `grep` )**:

```
grep "pattern" file.txt
```

Example: Search for the word “error” in all `.log` files:

```
grep "error" *.log
```

- **Recursively search for a pattern inside files**:

```
grep -r "pattern" /path/to/directory
```

## 3. File Size and Disk Usage:

- **Check the size of a specific file**:

```
du -h file.txt
```

Example output: `4.0K file.txt`

- **Check the size of a directory**:

```
du -sh /path/to/directory
```

Example output: `56M /path/to/directory`

- **Display disk space usage for all files and directories**:

```
du -h /path/to/directory
```

- **Display available disk space on mounted filesystems**:

```
df -h
```

Example output:

```
Filesystem  Size  Used Avail Use% Mounted on
/dev/sda1   50G 10G 38G  21% /
```

## 4. Miscellaneous Commands:

- **Display current working directory**:

```
pwd
```

- **Clear the terminal screen**:

```
clear
```

- **Show command history**:

```
history
```

- **Check system uptime**:

```
uptime
```

- **Show logged-in users**:

```
who
```

- **Show running processes**:

```
ps aux
```

- **Kill a process by PID**:

```
kill -9 PID
```

## 5. File Permissions:

- **Check file permissions**:

```
ls -l file.txt
```

Example output: `-rw-r--r-- 1 user user 1234 Sep 17 10:00 file.txt`

- **Change file permissions**:

```
chmod 755 file.txt
```

- **Change file ownership**:

```
chown user:group file.txt
```

These commands cover a range of useful functions for working with dates, times, file searches, sizes, and other system-related tasks in Bash. Let me know if you'd like more information on any of these!